

3>SHAKE

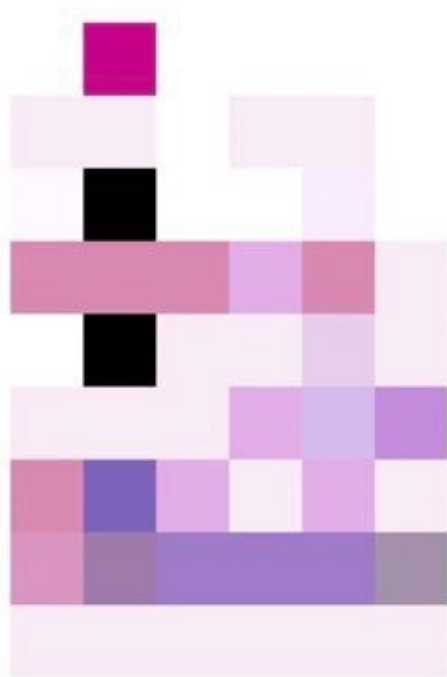
20分でわかる セキュアAPI

AIが対策を書く前に、許可条件と検証方法を決める

2026/08/07 『セキュアAPI』 翻訳者と考える、設計段階から組み込むセキュリティ

@nwiizo 20min

nwiizo



株式会社スリーシェイクのソフトウェアエンジニアです。格闘技、読書、グラフィアが趣味で、よく本を紹介しています。

技術書翻訳を手がけるたび、わかることが1つ増えるのと引き換えに、わからないことが3つ増えていきます。

単著『**おい、とりあえず終わらせる**』が刊行予定です。

翻訳書『**実践 プラットフォームエンジニアリング**』も刊行予定です。

インターネット上では **nwiizo** を名乗り、ブログ「**じゃあ、おうちで学べる**」を運営しています。X / GitHub もこのIDでやっています。

20分後にできること

AIが対策コードを書いても、許可条件が曖昧なら安全にはなりません。この20分では、**1つのAPIフローを実装前にレビューする方法**を説明します。

1. 守る境界を説明できる

誰が、どの対象を、どの順序で操作できるかを言葉にします。

2. AIとAPIの責任を分けられる

AIは操作を提案し、APIは入力形式と業務上の許可を検証します。

3. 判断を検証へつなげられる

脅威モデルの4問を、拒否するテストとログへつなげます。

明日、利用者 → AIエージェント → APIの1フローを図にし、4つの問いを書き出せます。



AIが対策を書く時代に本書を読む理由

生成AIにセキュリティコードを書かせ、AIエージェントにAPIを呼ばせるとき、**人が先に何を決める必要があるか**を整理します。AIは認可コード、テスト、ログ分析を生成できます。一方、AIエージェントは人に代わってトークンを使い、APIを高速に実行します。対策を実装できても、業務ルールがなければ「その操作が正しいか」は決められません。

AIが変えること

AIが短時間でできること

既知の攻撃への対策コードやテストを生成できます。

AIの導入で増えるもの

入力経路、API呼び出し回数、実行速度が増えます。

AIへ委ねる前に必要なもの

人とシステムが共有できる判定基準が必要です。

だから、人が先に決めること

① コードを書く前に、許可条件を決める

誰が、誰の対象へ、どの順序で操作できるかは、業務データから決めます。

② 入力元を信用せず、証拠を確かめる

生成AIも内部APIも入力元です。操作は提案できても、権限を与える側にはしません。

③ 生成結果を継続して検証する

起きてほしくないことを先に決め、契約・業務テスト・ログでAIの仕事も検証します。

AIにコードを書かせるほど、人は「何を許すか」と「正しく拒否できたか」を明文化する必要があります。



個別対策より、判断のメンタルモデルを更新する

では、その明文化は何があればできるのか。WAFのルール、JWTの検証、入力形式の検査は、それぞれ特定の失敗を止めます。より大切なのは、APIを見たときに「何を守るか、何が起き得るか、何をするか、十分だったか」を順に問うメンタルモデルです。

AIが速くする

個別の技術を学ぶ

用語、攻撃例、対策コードを質問し、理解できるまで説明してもらいます。

AIが速くする

実装候補を作る

仕様やコードを見せ、見落としの候補、テスト、修正案を作ってもらいます。

AIでは速くならない

問いを繰り返し使う

4つの問いと業務ルールを使い、AIの案が守る対象、失敗条件、検証方法を満たすか確かめます。

メンタルモデルは、1回の回答や1回の読書では変わりません。設計レビュー、拒否テスト、運用で見つけた乱用へ4つの問いを繰り返し使い、想定と結果が違った箇所を修正することで、時間をかけて更新します。

AIは個別の学習を速めます。本書は、対策を選ぶ前に何を問うかを繰り返し練習するために使います。



本書は脅威モデルから検証まで一体で扱う



『セキュアAPI 設計・構築・実装を貫く原則』

José Haro Peralta 著、株式会社スリーシェイク訳、翔泳社

脅威モデル Ch.1-3

何を扱い、何が起こり得るかを明らかにします。

防止 Ch.4-10

設計・実装・インフラの各層で脅威を止めます。

検知 Ch.11

ログ、メトリクス、トレースから未知の利用パターンを探します。

検証 Ch.12

脅威モデルから設計レビューとテスト項目を作ります。

4つの問い「何を扱うか、何が起こるか、どう対処するか、十分だったか」を、今日は許可条件・入力と権限の検証・対策の検証という3つの作業として説明します。

1. 許可条件

認証を通ったリクエストを、どこで拒否するか。

認証の成功は、認可の入力にすぎない

届いたリクエスト

GET /api/orders/456
利用者Aから届きました。



認証が答えたこと

送信者は確かに利用者Aである。
ここで認証の仕事は終わりです。



認可がこれから答えること

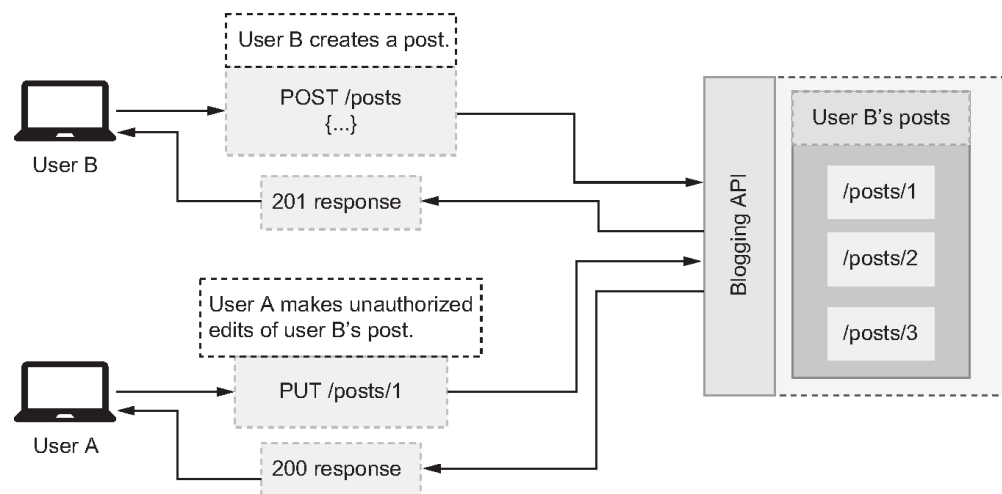
利用者Aは注文456を読んでよい
か。所有者、テナント（同じシステムを使う組織ごとの区切り）、
注文状態から判断します。

利用者Aがログイン済みでも、注文456が利用者Bのものなら拒否すべきです。主体がわかることと、主体と対象の関係が正しいことは別問題です。

関係を確認しないAPIは、認証を通ったリクエストにそのまま200を返します。



BOLAは「IDの問題」ではなく「関係の欠落」



User B: POST /posts → /posts/1
User A: PUT /posts/1 → 200 OK

同じ欠落は、注文以外のデータでも起きます。**BOLA**はBroken Object Level Authorizationの略で、データ1件ごとの認可の不備を指します。

リクエストを処理するコードは「投稿1が存在するか」を確認しています。しかし、**要求主体と投稿1の所有者との関係を確認していません。**

Figure 4.1 BOLA happens when user A can access resources or operations that should be accessible only to user B, such as updating user B's posts. より引用

存在確認は認可ではありません。所有者を見ないコードは、IDさえ合っていれば通します。

IDを推測しにくくしても、認可の代わりにはならない

BOLAへの対策としてよく挙がるのが「連番のIDをやめる」です。これは有効ですが、効く範囲が限られます。

発見を難しくする

UUIDなどの推測しにくいIDとレート制限は、他人のIDを総当たりで探す速度を落とします。ただし、一度漏れたIDが使われるのは防げません。

利用を拒否する

所有者・テナント・共有権限を、リソースへのすべてのリクエストで評価します。IDが正しく渡ってきても、関係が成り立たなければ拒否します。

前者は攻撃者が対象を**見つける**までの話で、後者は見つけた後に**使う**ときの話です。必要なのは後者です。

識別子は発見を難しくしますが、認可は見つかった後の利用を止めます。



認可判断は4要素でレビューする

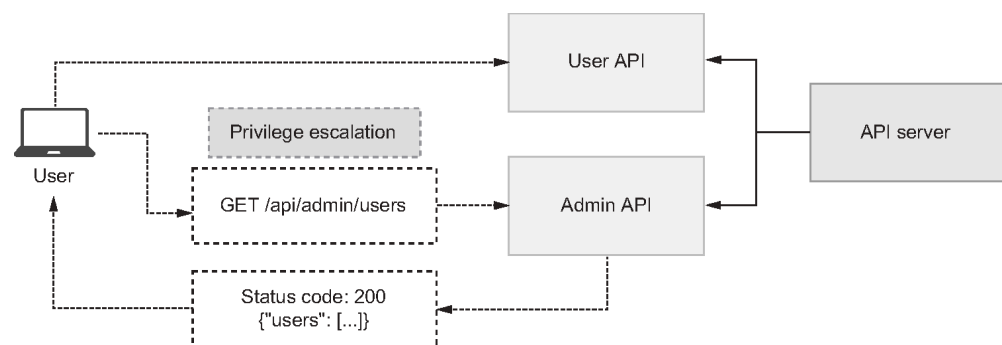


Figure 4.14 A common cause of BFLA is failure to check whether users' roles have access to the requested API. In this example, a normal user gets access to an admin API. より引用

主体

ユーザー、サービス、端末が主体になります。

操作

読む、更新する、承認するといった操作を評価します。

対象

注文、項目、管理機能などが対象になります。

文脈

所有者、テナント、状態、時刻を文脈として使います。

左の図は、一般ユーザーが管理者向けAPIを呼び出してしまう例です。主体は一般ユーザー、操作は読む、対象は管理者向けAPI、文脈はそのユーザーのロールにあたります。**4つのうちロールだけを見ていないため、200が返っています。**

4つのうち1つでも評価していなければ、通ってしまう経路が残ります。

ロールで足りるか、対象との関係が要るか

4要素のうち、ロールだけで決まる場合と、対象との関係まで見ないと決まらない場合があります。

RBAC ロールで決める

Role-Based Access Control。「管理者だけが実行できる」のように、主体のロールだけで可否が決まる場合に使えます。対象が誰のものは見ません。

ABAC 属性で決める

Attribute-Based Access Control。「所有者が、同じテナントで、未確定の注文だけを承認できる」のように、対象との関係や状態が必要な場合に使います。

どちらを使うかは好みではなく、**許可条件が業務データを参照するかどうか**で決まります。参照するなら、その判断は業務データを持つ場所でしかできません。

共通層で行う処理

共通層では、デフォルト拒否、トークン検証、粗いロール判定を行います。

業務APIで行う判断

ドメイン層では、所有関係、テナント、承認順序、例外条件を評価します。

証拠の検証は共通化できますが、業務上の許可は業務の事実がある場所で決めます。



脆弱性名が違ってても、欠けた問いは同じ

守るのは注文そのものだけではありません。認可の不備は1種類ではなく、**どの境界で許可を決めるか**によって、必要な設計とテストが変わります。

対象 BOLA（データ1件の認可不備）
この主体は、この注文を操作できるか。

項目 BOPLA（項目レベルの認可不備）
この項目を読み、または変更できるか。

機能 BFLA（機能レベルの認可不備）
この管理操作を実行できるか。

フロー 業務の乱用
正規機能でも、この順序や頻度を許すか。

利用者Aが注文456を読めるかは、**対象**の問題です。同じ注文でも、金額だけを隠すなら**項目**、返金を実行させるなら**機能**、短時間に同じ注文を何度も参照するなら**フロー**の問題になります。1つの注文APIに、4つの境界が同時に存在します。

「認証必須」で確認できるのは主体だけです。許可の可否は、主体 × 操作 × 対象 × 文脈でテストします。

2. 入力と権限の検証

AIも内部APIも入力元です。どこまで検証してから受け取るか。

ゼロトラストは通信元だけで入力を信用しない

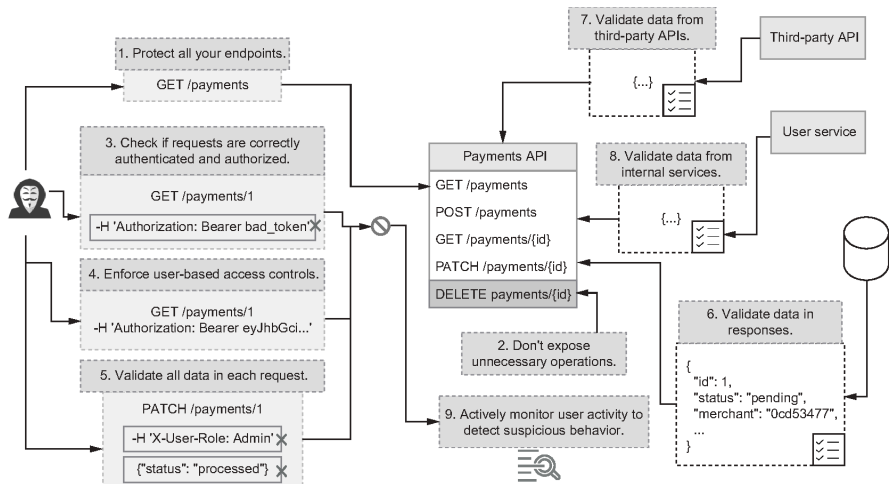


Figure 3.8 Zero-trust APIs apply NIST 800-207's principles to protect all assets and endpoints, apply robust access controls, and validate data across all flows while actively monitoring malicious activity. より引用

NIST SP 800-207は、ネットワーク上の場所だけを根拠に、暗黙の信用を与えないと整理します。

本書はこの原則をデータへ広げます。**リクエスト、レスポンス、DB、内部サービス、第三者API**を、それぞれの契約（型、必須項目、上限などの約束事）で検証します。

「内部APIだから」「自社DBだから」と検証を省くと、壊れたデータが別のAPIへ送信されたり、DBへ保存されたりします。ゼロトラストでは、**出所ではなく検証結果を根拠にデータを扱います。**

参考: NIST SP 800-207, Zero Trust Architecture

SSRFとは、サーバーが届く範囲を借りる攻撃

検証していない入力の中でも、URLはとくに危険です。SSRF（Server-Side Request Forgery）は、**サーバーに代理でURLを取得させる機能**を悪用します。「指定したURLの画像を取り込む」「入力された住所を外部APIで検証する」といった正規の機能が入口になります。

利用者から届く範囲

利用者のブラウザからは、社内ネットワークやクラウド内部の管理用アドレスには届きません。

サーバーから届く範囲

サーバーは内部ネットワークの中にいます。AWSのEC2には、内部アドレス `169.254.169.254` へ問い合わせると、そのサーバー自身の認証情報を返す仕組み（インスタンスメタデータ）があります。

この2つの範囲は同じではありません。攻撃者が狙うのは、その差です。

攻撃者は自分では届きません。届く場所にいるサーバーに、代わりに取りに行かせます。



WAFの設定不備が、AWS認証情報の流出につながった

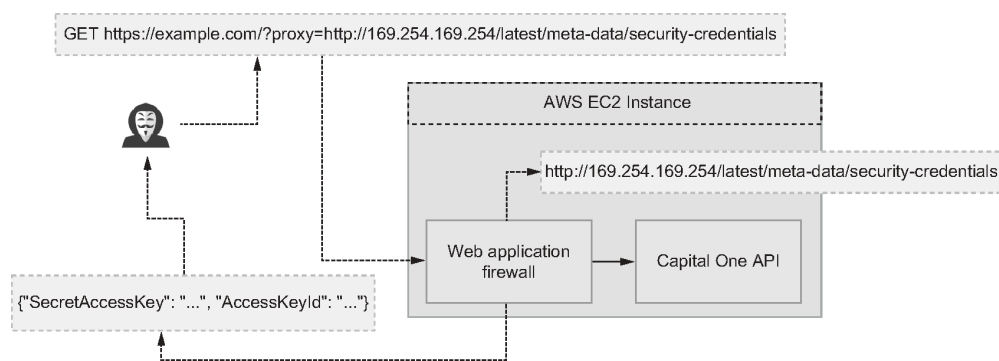


Figure 5.5 Capital One's SSRF attack happened due to a misconfiguration in its WAF, which allowed the threat actor to relay a request to Capital One's EC2 instance metadata endpoint and retrieve its AWS access keys. より引用

攻撃者がproxy URLを指定します。

↓
WAFが代理で取得します。

↓
EC2メタデータへ到達します。

↓
AWSアクセスキーが外部へ流出します。

2019年に米国の大手金融機関Capital Oneで起きた事例です。問題になったのはURL文字列だけではありません。外から到達できない場所へ行けるサーバー側の権限を、未検証の入力が操作しました。

正規のリクエスト1本で、外部からは触れないはずの認証情報が外へ出ました。

到達できる範囲を、3か所で小さくする

「取得先のURLを利用者が指定できる」機能そのものが必要な場合、SSRFの入口はなくせません。そこで、**入口をふさぐのではなく、届く範囲を狭めます。**

1. 入力

取得先を、あらかじめ決めた宛先だけに限定します。「危ないURLを弾く」拒否リストではなく、許可リストで絞ります。

2. 実行環境

取得処理を内部資源から隔離します。その処理を動かす場所から、内部の管理用アドレスへ届かないようにします。

3. 外向き通信

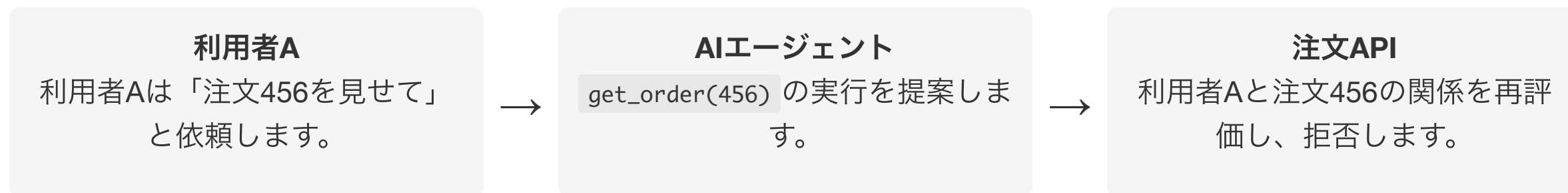
サーバーからの外向き通信を中継サーバーへ集め、そこで宛先を検査・制限します。

任意URLが必要ならSSRFは消せないため、サーバーが到達できる範囲を小さくします。



生成AIの出力だけでAPI操作を許可しない

入力元は、人と外部システムだけではなくになりました。AIエージェントは、人に代わってAPIを呼ぶ新しい入力元です。



責任の境界は、2つ目と3つ目の間にあります。左側でできるのは操作名と引数の**提案**までで、**許可の決定**は右側にしかありません。AIエージェントはこの境界を越えられません。

LLMは新しい入力元であって、権限の発行者ではありません。プロンプトに「管理者として実行」と書かれても、利用者と注文の関係は変わりません。

入力元が人からAIに変わっても、判定は同じです。変わるのは、同じ判定を求められる回数と速さです。

API Gatewayへ集約しても業務認可は残る

呼び出しの回数と速さが増えるなら、共通の制御は1か所へ集めたくになります。それを担うのがAPI Gatewayです。

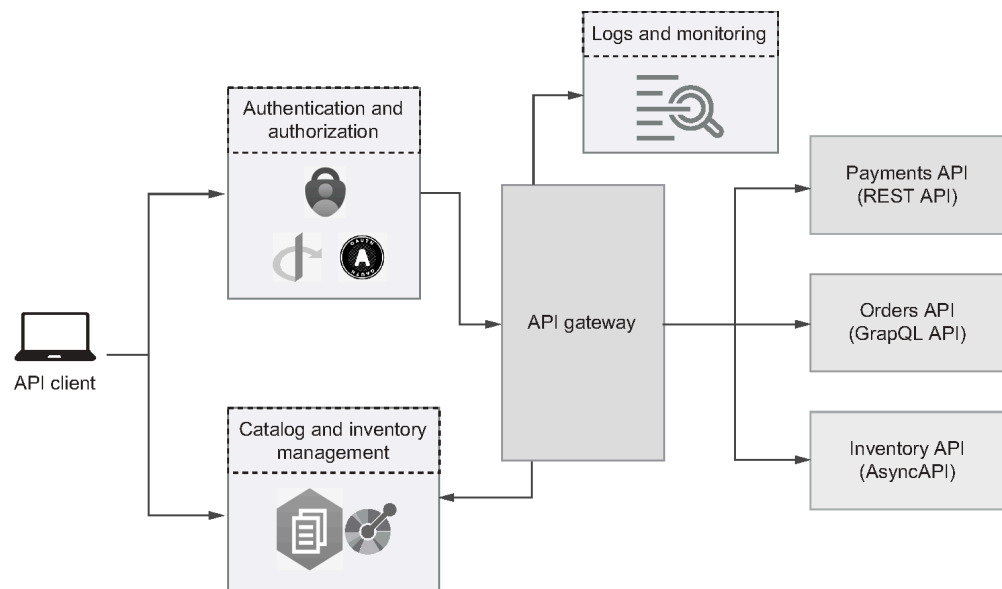


Figure 9.1 API gateways serve as single entry points to multiple backend APIs, helping us provide a unified API style, manage our API inventory, apply consistent access controls and security configuration, and improve observability. より引用

API Gatewayへ集約する処理

公開中のAPI一覧、トークン検証、レート制限、共通ヘッダー、ログ出力を集約します。

業務APIに残す判断

所有者、見せてよい項目、現在の状態、許される業務フローは、業務APIで判断します。

ただし、「誰の注文か」「今キャンセル可能か」という事実までは、API Gatewayは知りません。

業務データを見ずに判定できる処理だけを集約します。それ以外は業務APIに残ります。

3. 対策の検証

決めた許可条件が実際に効いていると、どうやって言えるか。

シフトレフトは判断の直後に検証する

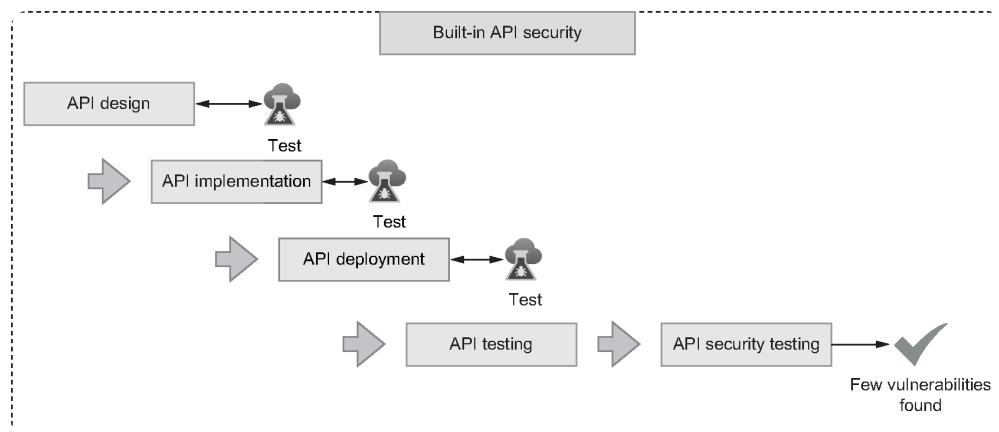


Figure 3.2 To build secure-by-design APIs, we must address security early in the SDLC and shorten the feedback loop by assessing our vulnerabilities frequently. より引用

完成後の診断は実装された表面を検査できます。しかし、所有関係や承認順序の誤りは、仕様通りに動くほど見つけにくくなります。

各判断の隣に検証を置き、問題を未解決のまま次の工程へ進めないようにします。

設計

脅威モデルと認可境界をレビューします。

実装

契約、認可、入力をテストします。

配備

設定とポリシーを検査します。

運用

乱用と拒否理由を観測します。

OpenAPIは入出力を定義し、業務認可は決めない

実装の段でまず書けるのは、入出力の約束事です。OpenAPIはそれを機械が読める形で書く仕様で、ここから検査もテストも自動生成できます。

仕様として表現できる

- エンドポイントとHTTPメソッドを定義します。
- 入出力の型、必須項目、上限を制約します。
- 認証方式と必要なスコープを宣言します。
- 仕様と実装のずれを検知できます。

業務の事実がなければ決められない

- 注文と利用者の所有関係は、業務データから判断します。
- 送金や承認で許される順序は、業務ルールで決めます。
- 自動化が乱用かどうかは、利用文脈から判断します。
- 例外時に受け入れる事業リスクは、責任者が決めます。

契約テストが証明するのは、**実装が仕様へ適合したこと**です。注文の所有者や承認順序という方針自体が正しいことまでは証明しません。

仕様に完全に適合したAPIでも、利用者Aに注文456を返せてしまいます。



3段階のテストで業務上の拒否を検証する

仕様への適合と、業務方針の妥当性は、別のテストで確かめます。段階を3つに分けます。

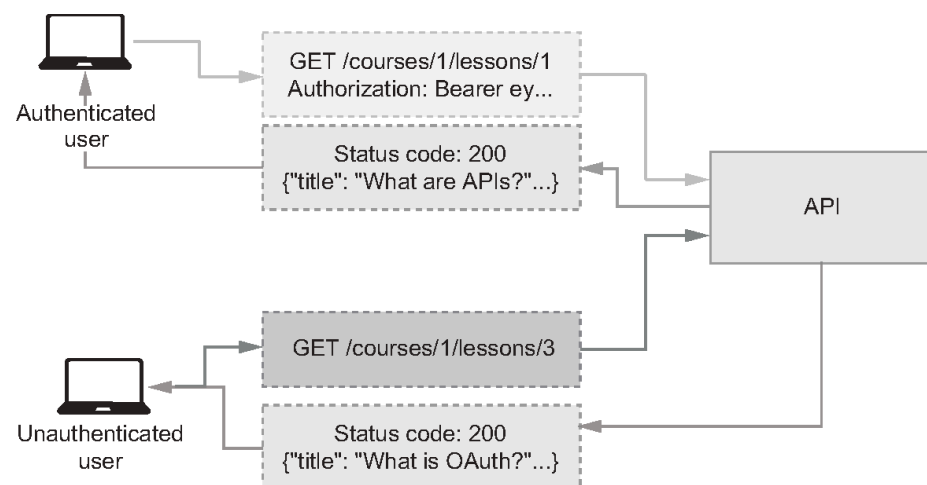


Figure 12.1 Our threat model considers the possibility that unauthenticated users can access protected content, so we must write tests to verify that our authentication layer works properly. より引用

1. 設計を静的に検査する

OpenAPIの構文・ルール検査を使い、曖昧な境界を減らします。

2. 仕様適合と想定外入力を検査する

契約テストと想定外入力の自動生成で、実装のずれを探します。

3. 業務上の拒否を確かめる

認証、BOLA、RBAC / ABAC、業務フローを複数の主体で試します。

3層目は準備コストが高い一方、汎用ツールだけでは確かめられません。AIエージェントなら、**主体・注文・金額・ツール実行順序**を変え、拒否される経路をテストします。

脅威モデルの「起きてほしくないこと」を、失敗するテストへ翻訳します。

ログから想定外の利用パターンを検知する

テストで確かめられるのは、起きると想定できたことだけです。想定できなかった使われ方は、動いているシステムからしか分かりません。

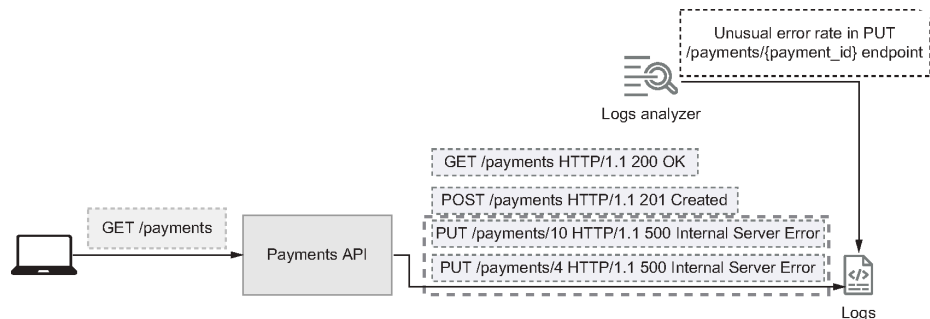


Figure 11.1 By continuously analyzing our logs, we gain an understanding of the state of our system. In this example, we find that one endpoint is encountering an unusual error rate. より引用

監視

監視では、既知の閾値を継続して確かめます。

オブザーバビリティ

残した文脈を使い、未知の問題を探索します。

監視は「壊れていないか」を確かめます。オブザーバビリティは「なぜそうなったか」を後から追えるようにしておくことです。

HTTP 403の件数だけでは、どの関係を拒否したか再現できません。「誰が何を許可・拒否されたか」を業務上の出来事として記録します。

認可の判断を、後から再現できる形で残す

残す情報

その判断をもう一度たどれるように、主体、操作、対象、許可 / 拒否の理由、処理を追跡するID、AIが呼び出したツール名を残します。

残さない情報

生のトークン、秘密情報、無制限なプロンプト本文は残しません。ログ自体が漏れたときの被害が、得られる情報より大きくなります。

この記録があると、運用中に見つけた乱用を**脅威モデル**へ追加し、それを拒否する**テスト**へ翻訳できます。設計時に想定できなかった経路が、ここで初めて手に入ります。

運用で見つけた乱用を使って、認可ルールとテストを更新します。

1つのAPIフローに4つの問いを書く

1. 何を扱っている？

利用者がAIエージェントへ依頼し、AIエージェントが注文APIを呼び出します。

2. 何が起き得る？

他テナントの注文へアクセスしたり、壊れた引数や同じ操作が送られたりします。

3. 何をする？

API側で認可と入力形式を検証し、重複実行と大量呼び出しを制限します。

4. 十分だった？

拒否を確かめるテストと、ログやメトリクスで検証します。

これはThreat Modeling Manifestoの4問です。大きな会議から始めず、変更する1フローを図にして、問い・対策・検証を同じ変更へ置きます。

1つのAPIフローから始め、許可条件と拒否条件をテストできる文章で残します。



本日のまとめ

許可条件

主体・操作・対象・文脈の関係と、業務の順序を評価します。

入力と権限の検証

内部APIや生成AIも例外にせず、入力形式と権限を毎回検証します。

対策の検証

脅威モデルから仕様と業務テストを作り、運用ログから更新します。

翻訳者として一番伝えたいこと

AIは既知の対策を提案し、コードやテストを生成できます。しかし、誰にどの操作を許すかは、プロダクト側で決める業務ルールです。そのルールが曖昧なら、AIは誤った許可条件を高速に実装します。本書の4つの問いを設計、テスト、運用で繰り返し使い、判断のメンタルモデルを時間をかけて更新します。

AIに対策を任せる前に、許可条件、入力の検証方法、拒否を確かめる方法を文章で定義します。



参考資料

- [Secure APIs](#) — Jose Haro Peralta, Manning Publications
- 『セキュアAPI 設計・構築・実装を貫く原則』 — Jose Haro Peralta 著、株式会社スリーシェイク訳、翔泳社
- [OWASP Top 10 API Security Risks – 2023](#)
- [NIST SP 800-207 Zero Trust Architecture](#)
- [Threat Modeling Manifesto](#)



The logo for 3 SHAKE, featuring the number '3' in white, a small colorful icon (yellow, green, and blue) to its right, and the word 'SHAKE' in white capital letters.

3 SHAKE

ありがとうございました

ご質問・ご相談はお気軽にお問い合わせください

@nwiizo | <https://3-shake.com>